AI駆動開発 大阪支部 Jev特別会 ・ 鈴山英寿（RAYVEN）

Jev の中身は 50行で再現できる

小さな LLM で Jev を自作し、本物と公開データ300問で比べたら、差は「自信の正しさ」にあった

▶ PRESS START

じこしょうかい – ステータス

鈴山 英寿（すずやま ひでひさ）／ RAYVEN CTO

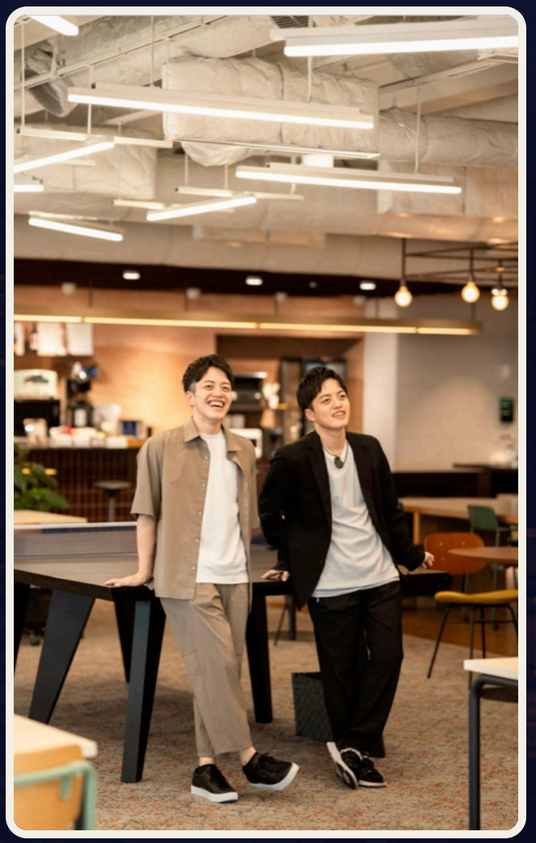
これまでの ぼうけん

学生時代　フリーランスエンジニア（スクールのメンター、受託開発）

2023.04　ヤフー入社。CTO 直下の SWAT チームで技術課題を解決

2023.10　LINEヤフー統合1年目に PayPayカードへ約1年出向

2024.07　RAYVEN を共同創業。開発から採用・営業まで



しゅみ　サウナ・テニス　　いま　VIVANT を一気見中

いまの クエスト　AI が働ける組織づくり。社内で Jev も使い始めた

ギルド しょうかい – 株式会社RAYVEN

RAYVEN

AIエージェントに、社員証を。

人は入社すると社員証をもらい、入れる部屋と使えるシステムが決まる。AI にも同じしくみを

AI の社員証を発行する

Tumiki

AI エージェントに専用の ID を渡し、触れてよいデータとツールを決める。どの AI が、誰の指示で、何をしたかを記録する

AI の配属先を見つける

WorkLens AI

PC 業務の記録から、AI に任せられる業務と減らせる時間を見つける。解析方法の選択に Jev を使用

AI の導入を決めるところから、安全に働かせるところまで。社内でも自分たちで使い、手続きと記録は AI、判断は人で回している

3 「AIエージェントにも、社員証を。」は CEATEC 2026 出展時のコンセプト

Jev は「選ぶだけ」の AI

文章を生成せず、選択肢ごとの確率（＝自信）を返す。公称で 193.6倍速く、444.6倍安い（TypeSafe AI）

この中身、自分で作れる？

LLM は「次の1語」を確率で予想しているだけ

入力した文章

日本の首都は

次に来る1語は？

LLM が出す「次の1語」の確率の表（全約15万語）

…… 残り約15万語にも、ごく小さな確率がつく

1語＝トークン（LLM が扱う文字のかたまり）。「東京」も「東」と「京」の2つに分かれる

チャットの返事は、この予想のくり返しでできている

1回目　日本の首都は東　← 69.5% でいちばん高い1語を書く

2回目　日本の首都は東京　← 91.5%

3回目　日本の首都は東京です　← 36.3%

4回目　日本の首都は東京です。　← 91.7%

ChatGPT や Claude の返事も、これを数百〜数千回くり返したもの

「考える」モデル（thinking）は、答える前に考える文章も同じ方法で書く。その分だけ遅い（実測：4B で 0.6秒 → 21.7秒）

6　Qwen3 4B の実測値。毎回いちばん確率の高い1語を選んだ場合

書かせずに、最初の1語の確率だけを覗く

いつものチャット

予想 → 書く → 予想 → 書く → ……

数百〜数千回くり返して、文章で答える

自信の度合いは、文章からは分からない

今回の自作（Jev 風）

予想を1回だけ。何も書かせない

確率の表から A / B / C の行だけ読む

→ 答えと「自信（確信度）」が数字で出る

→ 自信が低い答えだけ、人に確認を回せる

たとえるなら：相手が口を開く直前に「A と言いそう 7%、B と言いそう 87%」と心の中を覗いて、そこで止める

7 「考える」機能はオフにする。オンだと最初の1語が「考え始めの合図」になり、A/B/C が読めない

選択肢に記号を振り、LLM に1回だけ読ませる

LLM に渡すプロンプト（実物）

```
# 状況
お客様：昨日から本番環境にログインできません。今日の15時の商談デモで使う予定です。

# 質問
深刻度は？

# 選択肢
A. 低　B. 中　C. 高

最も適切な選択肢の記号を1文字だけ答えてください。
```

1. 選択肢に記号を振る

A / B / C は、どれも LLM にとって1トークン

2. 文章は生成させない

プロンプトを1回通す（forward）だけ。答えを書かせない

3. 次の1トークンを予想させる

LLM は「次に来る語」の確率を全語彙について出す

モデルは Qwen3 1.7B / 4B（4bit 量子化）を MLX で M4 Mac 上で実行。thinking は無効化

1回の予想で出てくるのは、約15万個の数字の列

モデルの出力（生の形）

(1, 97, 151936)

97 ＝ 入力したトークンの数

151,936 ＝ 語彙の数。1語ごとにスコアが1つ

使うのは最後の位置の1行だけ（＝次の1語の予想）

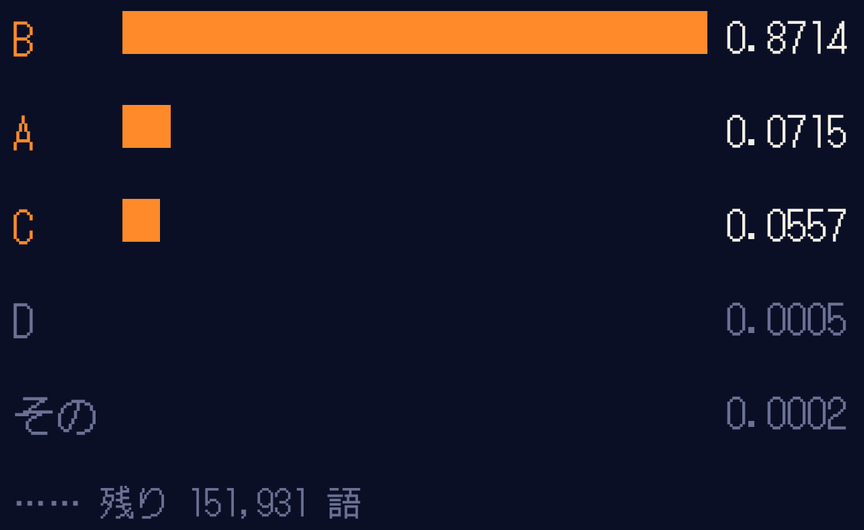
番号	1語	スコア	確率
33	B	30.12	87.14%
32	A	27.62	7.15%
34	C	27.38	5.57%
35	D	22.62	0.05%
57326	その	21.75	0.02%
……	……	……	……
50000	RESET	-3.58	ほぼ 0%

スコア（ロジット）を softmax で確率に直すと、全15万語で合計100%。スコアの差が 2.5 開くと確率は約12倍

9　自作 1.7B に深刻度の質問（A. 低 / B. 中 / C. 高）を入れたときの実際の出力。モデルの中では1語ずつ番号で扱われる

次トークンの確率から A/B/C だけ取り出す

次トークンの確率（全 151,936 語の上位）



A/B/C だけ残して、合計を1にする

A. 低　0.07

B. 中　0.87　← いちばん高い

C. 高　0.06

答え：中

確信度：0.87

10　自作 1.7B の実測値。A/B/C 以外の確率は合計 0.13% で、捨てても答えはほぼ変わらない

全部で53行。確率を取り出す核心は5行だけ

言語：Python（Apple の機械学習ライブラリ MLX）　ファイル：minijev.py の核心部分

```
logits = model(mx.array(tok.encode(prompt))[None])[0, -1]

full = mx.softmax(logits.astype(mx.float32))

picked = full[mx.array(LABEL_IDS[: len(options)])]

mass = picked.sum().item()

return dict(zip(options, (picked / mass).tolist())), mass
```

① プロンプトを1回だけモデルに通し、次の1語のスコアを全15万語ぶん受け取る

② スコアを確率に直す（全部足すと100%）

③ A・B・C の行だけを取り出す

④ A・B・C の確率の合計を出す

⑥ 合計で割って100%に直し、「選択肢 → 確率」の形で返す

装備：Python ＋ MLX ＋ Qwen3 1.7B / 4B（4bit）・M4 の Mac で動く ・ネット不要　github.com/hisuzuya/minijev

自作でつまずいた点は3つ（本物なら不要）

シンキングスライム

thinking を切らないと、最初のトークンが <think> になる

ふたごヘビ

「A」と「 A」は別トークン。読みラベルは1トークン。違えると確率を取りこぼす（合算しても今回は差なし）

26択ゴーレム

A〜Z の26択まで（Jev は255択）

動いた。でも、まちがえても自信満々

入力：在庫は りんご12個・みかん9個・ぶどう15個。注文は みかん10個。

勇者は「判定」を はなった！

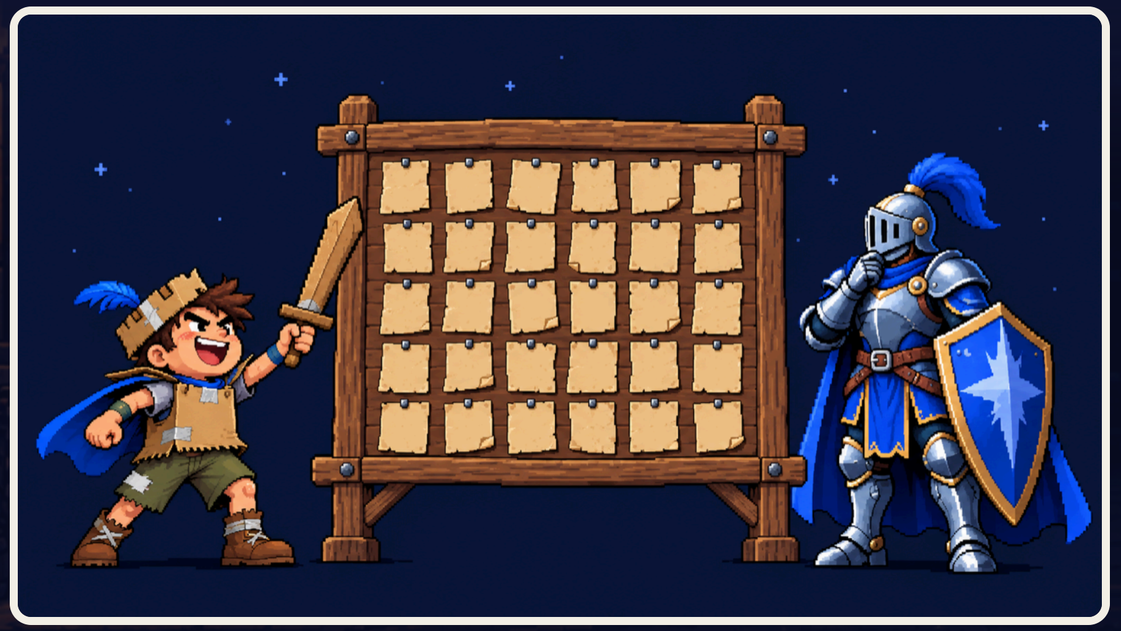
質問 …… 在庫だけで 注文に対応できる？

勇者の答え …… はい（確信度 1.00）

騎士の答え …… いいえ（確信度 0.96）

……しかし みかんは 9個しかない！ 勇者は 自信満々で まちがえた！

公開データ300問を、5者に解かせて比べる



クエスト（各50問）	何を見るか
WRIME	SNS 投稿の感情（3択）
livedoor ニュース	記事の分類（9択）
JNLI	文と文の含意関係（3択）
Jamp	時間表現の推論（3択）
JCommonsenseQA	常識（5択）
JMMLU	数学・ビジネス知識（4択）

パーティ：勇者 Lv1・Lv4・Lv8（自作 1.7B・4B・8B） ・ 騎士（本物の Jev）
賢者（Opus 5.5 に JSON スキーマで答えの形を固定させたもの）

14　公開されている日本語データセット6種から50問ずつ乱数で抜き出した。出典は26枚目。自作の問題（30問・100問）での結果は付録

正答率に加えて「確信度が当たっているか」を見る

業務での使い方：確信度 0.7 以上なら自動で処理、0.7 未満なら人に回す

① 正答率

何問当たったか。当てずっぽうなら300問中 約78問

② 外したときの確信度

外すときに迷えているか。高いまま外すと、人に回らず素通りする

③ 確信度どおりに当たるか

「0.99」なら本当に99%当たるか。成り立てば 0.7 で人に回せる

理想：確信度が高いほどよく当たり、外すときは迷っている

15 「人に回す・自動で誤り（事故）」の件数は ③ から決まるので付録 A4 に回した

正答率：自作は 8B でも 64%。Jev 82%、Opus 89%

勇者 Lv1（自作 1.7B）　137/300

勇者 Lv4（自作 4B）　178/300

勇者 Lv8（自作 8B）　192/300

騎士（本物の Jev）　245/300

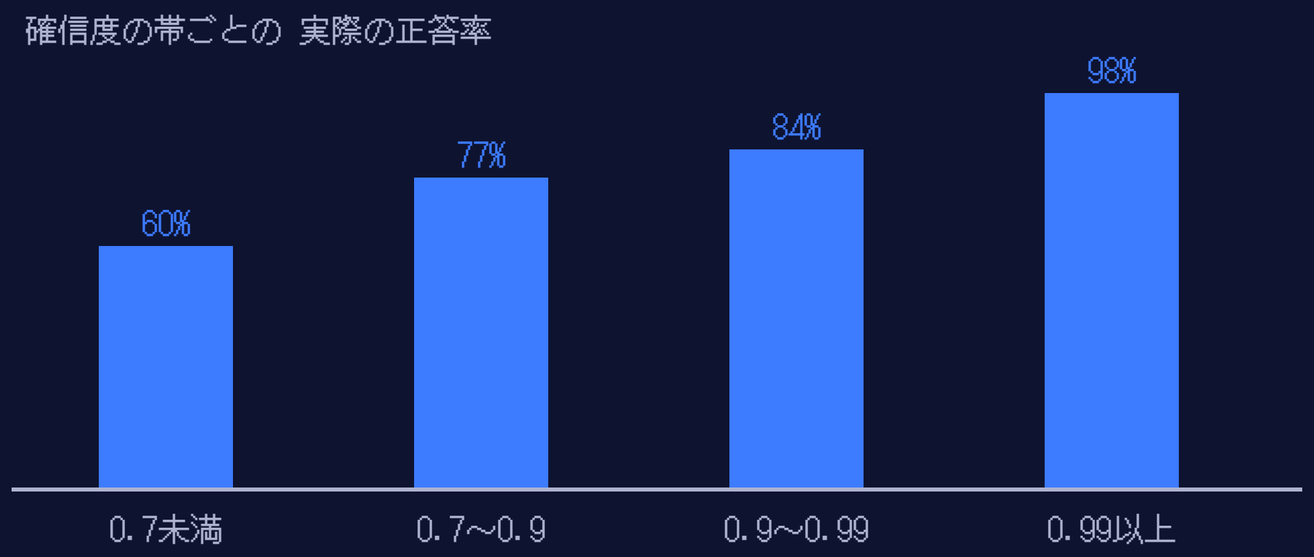
賢者（Opus 5.5 ＋ JSON 固定）　267/300

↑ 当てずっぽうなら 78

レベルを上げても 届かない。正答率でも 騎士が上

16　公開データ300問。差が大きいのは記事の分類・時間の推論・JMMLU（Jev は各 33〜43問、自作 8B は 21〜27問）

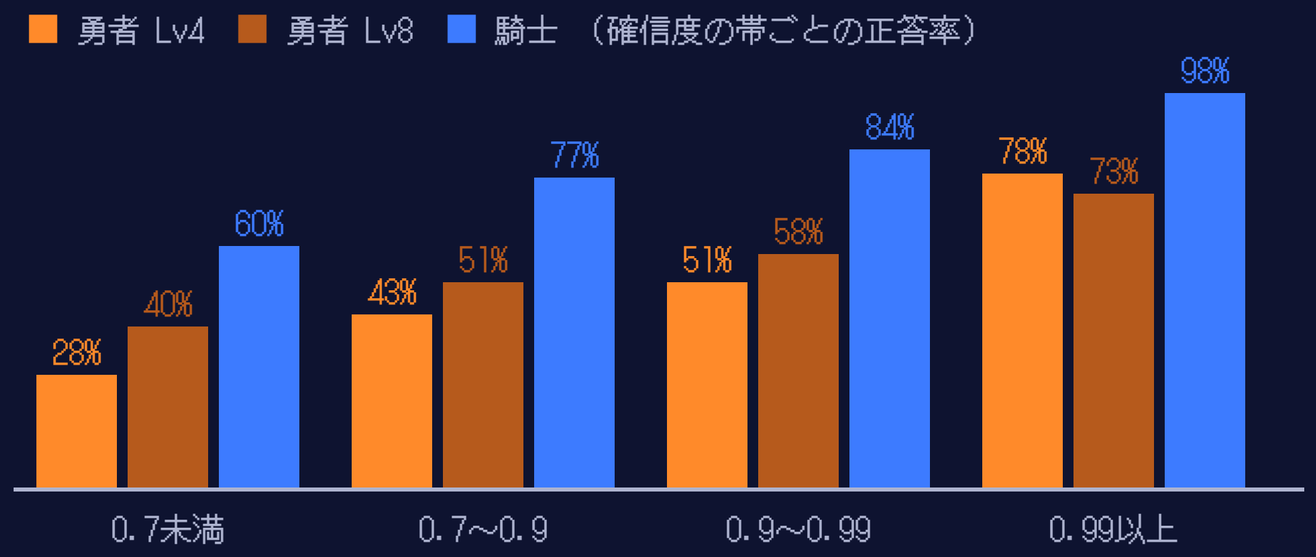
Jev が「0.99以上」と言った102問は、98% 正解



騎士は 確信度が高いほど よく当たる。
外すときは 迷っている（外した問題の確信度 中央値 0.67）

　公開データ300問。賢者（Opus）も同じ傾向で、0.99以上と言った13問は全問正解

自作が「0.99以上」と言っても、4〜5回に1回は外れる



勇者は 自信満々で ミミックに飛びこむ。
外した問題の 3〜4割が 確信度 0.99以上（騎士は 4%）

18 「0.99以上」の正答：自作 4B 154問中120問、8B 174問中127問、Jev 102問中100問。確信度で人に回す設計が成り立たない

実測：賢さは Opus。Jev は10問で30倍速く、桁違いに安い

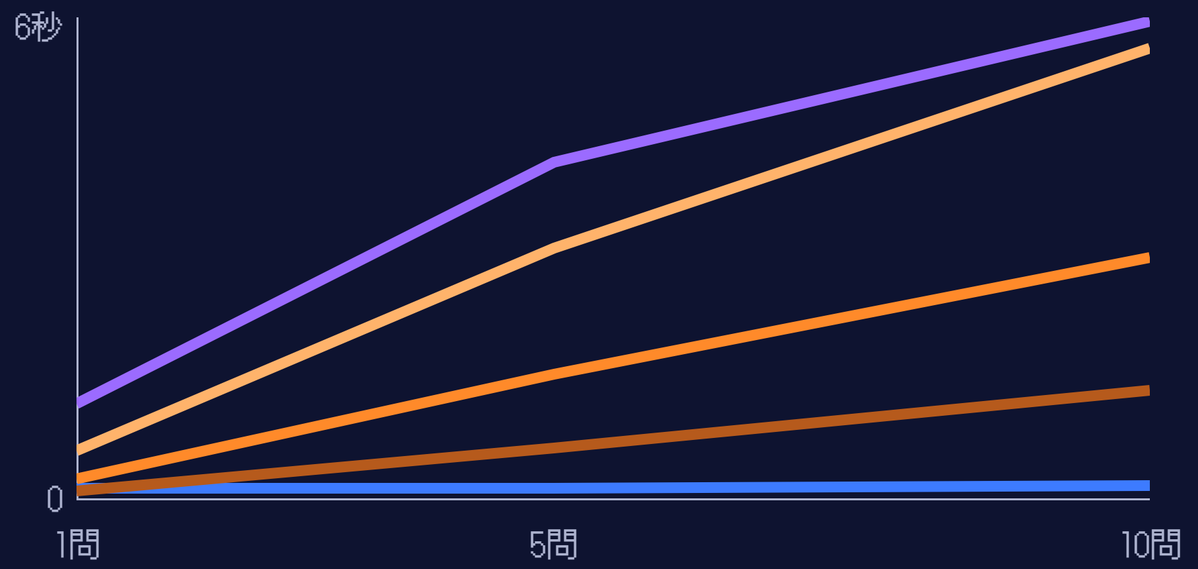
	騎士（Jev）	賢者（Opus 5.5）
正答（300問）	245（82%）	267（89%）
0.99以上での正答	98%（102問）	100%（13問）
1問の応答	0.16 秒	2.3 秒
10問まとめて	0.19 秒	6.0 秒
1問の費用（目安）	約 $0.00002	約 $0.026

賢さなら賢者。
大量の判定を速く安く回すなら騎士。

19　Opus は Claude Code CLI の --json-schema（effort low）で実行。確信度は自己申告。時間は API 時間、費用は CLI の定価見積もり。Jev の費用は公称単価（入力 $0.042/100万トークン・出力無料）× 実測381トークンの概算

第3章　たたかう ・ ROUND 3 質問を増やしたとき – 勇者は 息が きれてきた！

10問まとめて聞いても、Jev だけ 0.2秒のまま



10問のとき：騎士 0.19秒　賢者 6.0秒　勇者 Lv1 1.4秒 ・ Lv4 3.1秒 ・ Lv8 5.7秒

勇者は 質問が増えるほど 息切れ。騎士は 何問でも 涼しい顔。

20　線は実際の応答時間と同じ速さで伸びる。同じ状況に質問を1・5・10問（3回の中央値）。条件は同じではない（騎士は API 往復込み、勇者は M4 ローカル、賢者は API 時間）

選択肢の順番を変えると、自作は答えがぶれる

勇者 Lv1（自作 1.7B）

29%

86 / 300問

勇者 Lv4（自作 4B）

54%

162 / 300問

勇者 Lv8（自作 8B）

59%

176 / 300問

騎士（本物の Jev）

94%

282 / 300問

勇者は 順番を変えられるだけで、半分近く まどわされた。
騎士は ほぼ ぶれない。

21　公開データ300問で、選択肢の順番を3通りに入れ替えても答えが変わらなかった割合。賢者（Opus）は今回は未測定

自作は正答率でも自信でも及ばない。Jev は速くて安い

ステータス	勇者 Lv4（自作）	騎士（Jev）	賢者（Opus）
正答率（300問）	× 59%	○ 82%	◎ 89%
0.99以上での正答	× 78%	◎ 98%	◎ 100%
10問まとめて	× 3.1秒	◎ 0.19秒	△ 6.0秒
並び順への強さ	△ 54%	◎ 94%	– 未測定
1問の費用（目安）	◎ 0円（手元）	◎ 約$0.00002	△ 約$0.026
データを外に出さない	◎ 手元で完結	× API	× API

53行で再現できたのは「仕組み」まで。正答率と自信の正しさは、学習の差と考えられる

　Jev の内部構造は非公開。並列評価は公式仕様、学習は RLCD と説明されている。それ以上は推測

読んでいるのは「次に書く記号」の確率。当たる確率ではない

1. 確率の意味がちがう

確率は「次にこの記号を書きそうか」の強さ。答えが当たる見込みを測るようには作られていない

2. 言い切るよう調整済み

対話向けの追加学習で、確率が1つの候補に集中しやすい。外した問題の3〜4割が 0.99 以上

3. 割り直しで さらに高く

A・B・C だけで100%に割り直す。1.7Bでは3つの合計が 1.7% の問題もあった。迷いが外に逃げ、割ると消える

Jev は外した問題で確信度が下がっていた（中央値 0.67）。
自作で近づけるなら、正解付きのデータで確率を補正する

　1.7% は自作の問題での実測。補正（温度スケーリングなど）は今回は未検証

使い分け：試作は自作、大量は Jev、難問は Opus

勇者（自作）
試作・機密データ・オフライン。確信度は閾値に使わない

騎士（本物の Jev）
大量の判定を速く安く。確信度で人に回す本番運用

賢者（Opus ＋ JSON 固定）
件数が少なく、正確さ最優先の判定。理由も欲しいとき

「答え」は 大きくすれば 少しずつ近づく。
「自信の正しさ」は 近づかなかった。

注意：公開データ300問・各1回の実験です

- 公開データ6種x50問・各1回。自作の問題（30問・100問）でも結論は同じ（付録）
- 公開データは、各モデルの学習データに含まれている可能性がある
- 勇者は確信度の補正（温度スケーリングなど）を試していない
- 速度は条件をそろえていない（騎士は API 往復込み、勇者は手元の M4）
- 賢者は Claude Code CLI 経由。確信度は自己申告、費用は定価の目安

つづく：補正の装備で、勇者はどこまで強くなるか

コード：github.com/hisuzuya/minijev

ぼうけんのしょ – 出典

使ったデータセット（公開データ6種）

WRIME　Kajiwara et al., NAACL 2021／Suzuki et al., LREC 2022（CC BY-NC-ND 4.0）
github.com/ids-cv/wrime

livedoor ニュースコーパス　株式会社ロンウイット配布（CC BY-ND 2.1 JP）
rondhuit.com/download.html

JGLUE（JNLI・JCommonsenseQA）　Kurihara, Kawahara, Shibata, LREC 2022（CC BY-SA 4.0）
github.com/yahoojapan/JGLUE

Jamp　Sugimoto, Onoe, Yanaka, ACL 2023 SRW（CC BY-SA 4.0）
github.com/tomo-vv/temporalNLI_dataset

JMMLU　Yin et al., SICon 2024（CC BY-NC-ND 4.0）。元は MMLU: Hendrycks et al., ICLR 2021
github.com/nlp-waseda/JMMLU

　各データセットから50問ずつ乱数で抜き出して使用。問題文は再配布していない（公開リポジトリには結果の数字だけを置いている）

まとめ – ぼうけんの きろく

今日の持ち帰り3つ

1. 作れた

LLM は次の1語の確率を出しているだけ。書かせずに確率を読めば、53行で Jev 風の「選ぶ AI」になる

2. でも差があった

自作は 8B でも正答率 64%（Jev 82%）。「0.99以上」と言っても4〜5回に1回は外れ、自信で人に回せない

3. 使い分ける

Jev の価値は「自信の正しさ」と速さ・安さ。試作は自作、大量は Jev、難問は Opus

ご清聴ありがとうございました

鈴山 英寿（RAYVEN CTO）

RAYVEN

次回の LT 会

AI Talks Night #9

11/9（月）20:00〜

オンライン・参加無料

WE'RE HIRING！

AI 好き・登壇好きの
なかまを募集中

← Wantedly

コードは こうかい中！

github.com/
hisuzuya/minijev

コード・評価データ・
結果・スライド PDF

付録1 – 自作の問題（100問）での結果

自作の問題（100問）でも、結論は同じだった

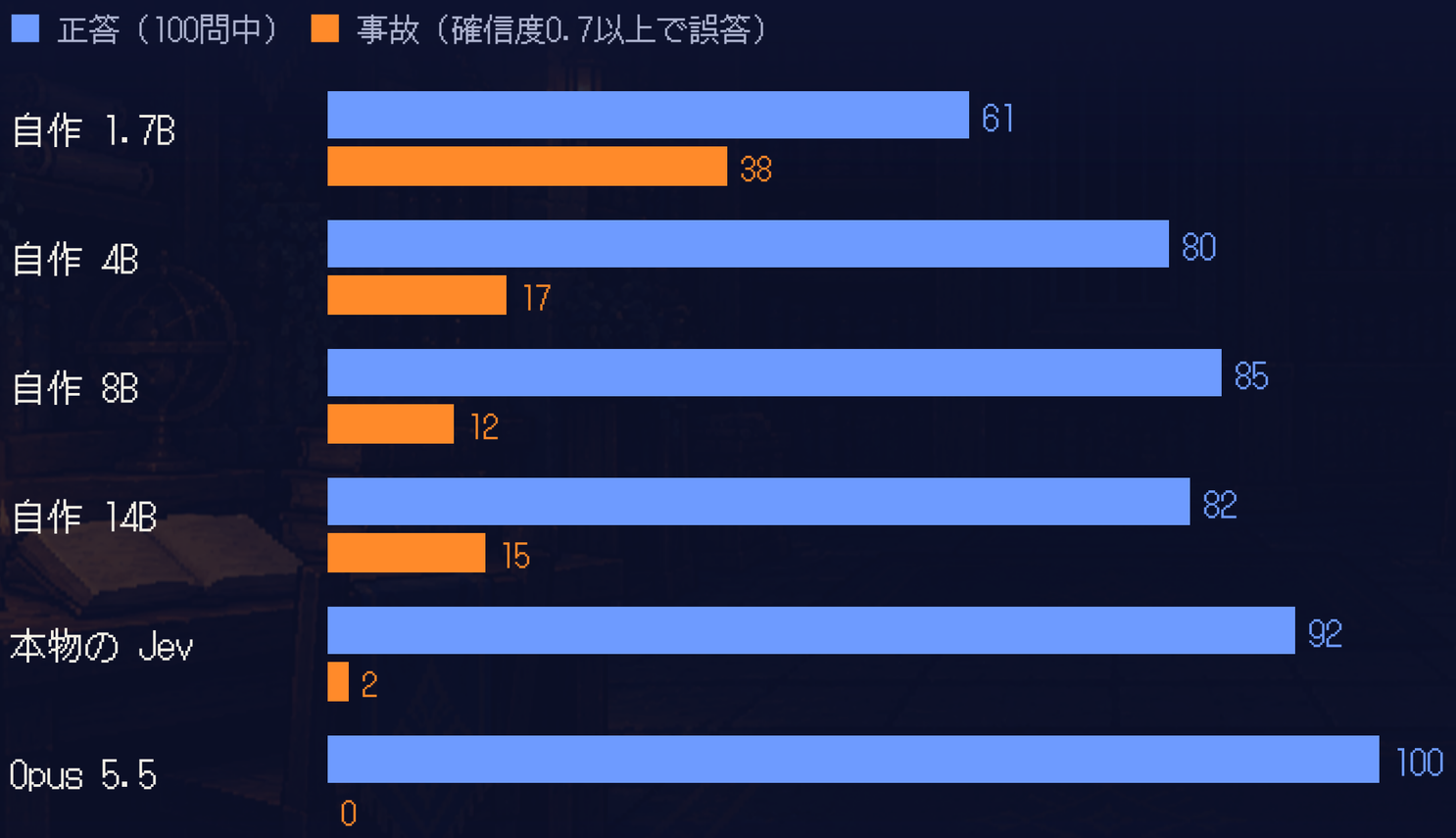
	重み	正答（95%CI）	事故	外したときの確信度	10問まとめて
自作 1.7B	0.9GB	61%（51〜70）	38	1.00	3.5秒
自作 4B	2.1GB	80%（72〜88）	17	1.00	5.8秒
自作 8B	4.3GB	85%（78〜92）	12	1.00	12.9秒
自作 14B	7.8GB	82%（74〜89）	15	1.00	16.3秒
本物の Jev	API	92%（87〜97）	2	0.61	0.17秒
Opus 5.5	API	100%	0	–	3.3秒

30問では「4B は Jev に迫る」と言えたが、100問では一段下（80% と 92%、信頼区間はほぼ重ならない）。自作の誤答は大きさに関係なく確信度 1.00

A1　事故＝確信度0.7以上で誤答（自動処理されてしまう）。でたらめに選ぶと 34.8/100。M4 Mac・4bit、2026-10-29

付録2 – 自作の問題（100問）で勇者を大きくしてみた

大きくしても「自信満々で外す」は治らない



正答：1.7B→4B で大きく伸び、その先は 85・82 で頭打ち。14B でも Jev に届かない

事故：4B〜14B で 12〜17件のまま。0.99以上の誤答は 14B でも13件

速さ：14B は10問で16秒。Jev の約100倍遅い

A2　自作は mlx-community/Qwen3-*-4bit。14B は 7.8GB、M4 32GB で実行

付録3 – 自作の問題（100問）で どこで つまずくか

自作がつまずくのは「数値」と「日付」

	基本（33）	否定・複合（17）	数値（17）	日付（17）	多選択肢（16）
自作 1.7B	27	13	3	6	12
自作 4B	33	14	11	8	14
自作 8B	32	16	9	12	16
自作 14B	32	16	10	9	15
本物の Jev	33	17	13	13	16
Opus 5.5	33	17	17	17	16

基本と多選択肢は 4B 以上でほぼ満点。誤りは税込計算・合計・割合・単位換算と曜日・営業日・期限の前後に集中。1トークンで答える方式では、計算の途中を書けないため

A3　詳細は minijev の results/v2/SUMMARY.md と results/SCALE.md。数値は2026-10-29の1回分

付録4 – 確信度 0.7 で区切った内訳

0.7 で運用すると、事故は 自作 83〜90件・Jev 26件

公開データ300問	正答	◎ 自動で正解	○ 人に回す	× 事故
勇者 Lv1（自作 1.7B）	137	130	24	146
勇者 Lv4（自作 4B）	178	163	54	83
勇者 Lv8（自作 8B）	192	180	30	90
騎士（本物の Jev）	245	202	72	26
賢者（Opus 5.5 ＋ JSON）	267	229	55	16

Jev も事故は ゼロではない。ただ 自作の 3分の1以下

A4　確信度0.7以上を自動処理、未満を人に回したときの内訳。事故＝自動処理したのに誤答。本編の「確信度の帯ごとの正答率」から決まる